Jakarta Air Quality: Basic Data Preprocessing

Fachry Alfareeza

January 2024 observations

1 What I wanted to do

I made this notebook to combine air quality readings with weather data for Jakarta. The main measurement is $PM_{2.5}$, which describes fine particles in the air. I wanted to see what happens during cleaning and merging, so the notebook shows sample rows, row counts, and graphs after each important step.

I chose these sources because they have different formats and some readings need to be checked before they can be used together. The project covers 1 January 2024 at 00:00 up to, but not including, 1 February 2024 at 00:00 in Jakarta time. The selected area is longitude 106.689–106.978 and latitude −6.370–−6.079.

2 Data and program design

OpenAQ and PurpleAir provide air measurements. Meteostat provides weather observations and station information. The starting data contain 3,990 OpenAQ rows, 2,906 PurpleAir rows, and 2,976 weather station-hours. Five nearby weather stations were listed, and four had observations. A station-hour means one hour from one weather station.

The notebook uses pandas for tables and Matplotlib for graphs. Requests and Meteostat support the download functions. Saved API responses and weather CSV files are used by default, so the notebook can run without new API requests. The original responses are kept unchanged.

The cells follow this order: load the data, clean each source, combine the air measurements, group them by hour, select a weather station, merge the tables, and check correlations. I used basic operations such as `drop_duplicates()`, `concat()`, `groupby()`, and `merge()`. Each step that changes the data shows a preview, summary, or graph using `head()`, `display()`, or Matplotlib.

3 Cleaning the air measurements

For OpenAQ, I remove exact duplicate rows, convert time and $PM_{2.5}$ to the correct types, and keep valid times and $PM_{2.5}$ values between 0 and 1,000 $\mu g/m^3$. Missing values cannot pass this check. This reduces the table from 3,990 to 3,254 rows. The 736 removed records are missing or invalid records, not 736 detected outliers.

PurpleAir has two channels, A and B. Both must be between 0 and 1,000. They must also satisfy

$$|A - B| \leq \max\left(10,\ 0.5\frac{A+B}{2}\right).$$

I use the mean of the accepted channels as $PM_{2.5}$. This keeps 2,905 of the original 2,906 rows. The range and channel checks are simple project choices, not a local calibration or a health standard.

Before-and-after boxplots show how the distributions change. Points outside the whiskers are still shown; they are not automatically deleted. High readings can be real pollution events. The boxplots can look similar even when many missing rows are removed because missing values are not plotted.

4 Combining air and weather data

I combine the air tables using the same fields: source, site ID, latitude, longitude, time, and $PM_{2.5}$. Rows with missing required fields are removed, and only the chosen area and period are kept. I then group existing readings by source, sensor location, and UTC hour and take their mean. No missing hours are added or filled. In this run, filtering and hourly grouping leave 6,159 rows.

For weather, I remove exact duplicates and convert the measurements to numbers. Values outside the following ranges become missing: temperature −30 to 60 °C, humidity 0–100%, rain 0–500 mm, wind speed 0–250 km/h, and pressure 850–1,100 hPa. Missing weather values are left unfilled.

The nearest available station to the Jakarta centre is 96745, Jakarta / Observatory, with 744 hourly records. I join its weather to each air measurement by UTC hour. The merge checks that each air row matches at most one weather row. It changes the table from 6,159 rows and 6 columns to 6,159 rows and 12 columns. All final columns have zero missing values in this saved run.

5 Results and before-and-after comparison

Stage	Air rows
Starting data	6,896
After cleaning	6,159
After area and time filtering	6,159
After hourly grouping	6,159
After weather merge	6,159

The program removes 737 air records overall, about 10.69%. Most of the change happens during cleaning. The merge adds weather columns without increasing the number of air rows.

I calculate Pearson correlations using one row per weather hour, so weather values are not counted repeatedly for different air sensors. The program would remove a later weather column if its absolute correlation with a retained earlier column was greater than 0.95. No columns are removed. Temperature and humidity have a strong negative correlation, about −0.95 when rounded, but the unrounded magnitude is below the cutoff.

Source	Count	Mean	Median	Maximum
OpenAQ	3,254	32.25	26.85	298.00
PurpleAir	2,905	31.02	23.55	551.60

These $PM_{2.5}$ values are in $\mu g/m^3$. Both sources have means above their medians, consistent with high readings pulling the average upward. Their maxima are much higher than their medians. Different sensor and time coverage means the difference between sources does not establish sensor bias.

The notebook has four graph outputs: before-and-after boxplots, a row-count chart, a weather correlation heatmap, and hourly $PM_{2.5}$ means with the number of readings in each mean. The count plot helps explain whether a changing mean also involves changing coverage. Copies of these graphs are included in the project folder.

6 Conclusion and limits

The final table contains 6,159 rows and 12 columns. The program makes the formats consistent, removes missing or invalid air readings, and adds weather from the same hour. Showing results after each step makes the process easier to check.

This basic version does not use interpolation or PCA. The correlation check is included, but it does not reduce the number of columns in this run. One weather station gives city-level context rather than the exact conditions at every sensor. High readings may need further investigation, and the results show patterns rather than causes.

Source links

- OpenAQ: `https://docs.openaq.org/`
- PurpleAir: `https://api.purpleair.com/`
- Meteostat: `https://dev.meteostat.net/python/api/meteostat.hourly`